STUDY NOTES — DEVELOPMENT METHODOLOGY

Instruction-Driven *Development (IDD)*

A structured methodology for AI-assisted software development — also known as Specification-Driven Development (SDD). Move beyond ad-hoc prompting to a disciplined, reproducible workflow where instructions are the blueprint and AI is the builder.

Updated: **April 2026**

Version: **1.0**

Category: **Foundation**

Reading Time: **~18 min**

Author: **Michaël Bettan**

01 The Problem with Ad-Hoc Prompting

Most developers using AI coding assistants today work in an **ad-hoc, reactive mode**: they open a chat, type a prompt like "build me a login page," accept the output, and move to the next thing. This works for simple tasks but breaks down for anything complex, team-based, or production-grade.

The result? **Inconsistent outputs**, lost context between sessions, no reproducibility, and code that works in isolation but fails when integrated. IDD solves this by treating AI interaction as a *structured engineering process*, not a casual conversation.

Inconsistency

The same prompt produces different results across sessions, models, or team members. No deterministic outputs.

Context Loss

AI forgets previous decisions. Each session starts from zero. Architectural coherence degrades over time.

No Collaboration

Prompts live in one developer's head. No shared understanding, no review process, no handoff protocol.

THE AD-HOC SPIRAL

Without structure, each AI interaction accumulates subtle inconsistencies. By the time you notice, your codebase is a patchwork of individually-correct but collectively-incoherent components. This is the **"death by a thousand prompts"** anti-pattern.

02 What is IDD / SDD?

Instruction-Driven Development (IDD) — also called **Specification-Driven Development (SDD)** — is a methodology where you write detailed, structured instructions *before* engaging the AI. These instructions serve as the contract between human intent and AI execution.

Think of it as **TDD (Test-Driven Development) but for prompts**: instead of writing tests first then code, you write specifications first then let AI generate the implementation. The spec is the single source of truth — versioned, reviewed, and shared across the team.

1

SPEC = SOURCE OF TRUTH

3×

MORE REPRODUCIBLE

60%

REWORK REDUCTION

TEAM-SCALABLE

Core Philosophy of IDD

- **Spec First, Code Second:** Write the instruction document before touching the AI — clarify what you want before asking for it
- **Instructions as Artifacts:** Specs are version-controlled, peer-reviewed, and reusable — they're as important as the code itself
- **Deterministic Intent:** The same instruction should produce functionally equivalent results regardless of when, where, or by whom it's executed
- **Separation of Concerns:** Human handles *what* and *why*; AI handles *how*
- **Iterative Refinement:** Specs evolve through feedback loops — each iteration makes instructions more precise and AI output more reliable

IDD VS SDD — SAME THING, DIFFERENT NAMES

The community uses both terms interchangeably. **IDD (Instruction-Driven Development)** emphasizes the instructional nature of prompts. **SDD (Specification-Driven Development)** emphasizes the formal specification. In practice, they describe the identical workflow.

03 The IDD Workflow (Step-by-Step)

IDD follows a disciplined, phased workflow. Each phase has clear inputs, outputs, and quality gates. The methodology can be applied to a single feature or an entire project.

PHASE 1

Define

Write the instruction document with requirements, constraints, and success criteria

PHASE 2

Review

Peer review the spec for clarity, completeness, and feasibility

PHASE 3

Execute

Feed the instruction to the AI and generate the implementation

PHASE 4

Validate

Compare output against spec. Run tests, review code, check constraints

PHASE 5

Refine

Update spec based on learnings, iterate until output meets all criteria

PHASE 6

Archive

Version the spec alongside code. Document patterns for reuse

Detailed Phase Breakdown

01 **Define the Instruction:** Write a structured document containing: project context, functional requirements, non-functional requirements (performance, security), technical constraints (language, framework, patterns), file structure expectations, test requirements, and acceptance criteria.

02 **Review & Validate the Spec:** Have a teammate or second AI review the instruction for ambiguity, missing edge cases, contradictions, and completeness. Fix before executing.

03 **Execute with AI:** Feed the instruction to your AI assistant. Use the spec as the primary context. Include relevant existing code as additional context. Ask the AI to explain its approach before generating.

04 **Validate Output Against Spec:** Check every acceptance criterion. Run generated tests. Verify file structure matches spec. Check for security issues. If output diverges, determine if the spec or the output needs adjustment.

05 **Refine the Instruction:** Based on what the AI got right and wrong, update the spec to be more precise. Remove ambiguity. Add constraints for areas where the AI made poor choices. This creates a feedback loop that improves future results.

06 **Archive & Reuse:** Commit the instruction alongside the code. Tag it with version. Create a template library of proven instructions for common patterns.

THE GOLDEN RULE OF IDD

If your instruction is ambiguous, the AI's output will be unpredictable. The quality of your specification directly determines the quality of the generated code. **Invest time in the spec, save time in debugging.**

04 Instruction Documents (Spec Anatomy)

The instruction document is the heart of IDD. It's not just a prompt — it's a **structured specification** that gives the AI everything it needs to produce consistent, high-quality output. Here's the anatomy of a well-crafted IDD instruction.

MARKDOWN — IDD INSTRUCTION TEMPLATE

```
# Instruction: [Feature Name]
## Version: 1.0 | Date: 2025-07-15 | Author: [Name]

## 1. Context
Project: [Project name and brief description]
Current State: [What exists, what's the starting point]
Goal: [What this instruction should produce]

## 2. Functional Requirements
- FR-01: [User can register with email and password]
- FR-02: [System sends verification email on registrati
- FR-03: [User can login and receive a JWT token]

## 3. Non-Functional Requirements
- NFR-01: Response time < 200ms for auth endpoints
- NFR-02: Passwords hashed with bcrypt (cost factor 12)
- NFR-03: Rate limiting: 5 login attempts per minute pe

## 4. Technical Constraints
- Language: Python 3.12
- Framework: FastAPI
- Database: PostgreSQL via SQLAlchemy
- Auth: JWT with PyJWT
- Testing: pytest with >80% coverage

## 5. File Structure
src/
  auth/
    router.py     # API endpoints
    service.py    # Business logic
    models.py     # SQLAlchemy models
    schemas.py    # Pydantic schemas
  tests/
    test_auth.py  # Unit + integration tests

## 6. Acceptance Criteria
- [ ] All FR items implemented and tested
- [ ] All NFR constraints verified
- [ ] No security vulnerabilities (no hardcoded secrets
- [ ] Code passes ruff linting with zero warnings
- [ ] README updated with setup instructions
```

Context Block

Gives the AI project awareness — what exists, what's the goal, and what constraints are inherited from previous decisions. This prevents the AI from making incompatible choices.

Functional Requirements

Numbered, testable statements of what the system must do. Each FR should be independently verifiable. Use "system shall" or "user can" language.

Non-Functional Requirements

Performance, security, scalability, and quality constraints. These are the guardrails that prevent the AI from taking shortcuts.

Technical Constraints

Locked-in technology choices: language version, framework, database, testing framework. Eliminates the AI's tendency to suggest whatever it "prefers."

File Structure

Explicit directory and file layout. Prevents the AI from creating its own (often inconsistent) project structure. Ensures the output fits into the existing codebase.

Acceptance Criteria

Checklist of verifiable conditions for "done." These are your quality gates — if any are unmet, the output needs refinement.

05 IDD vs Vibe Coding vs Traditional

Vibe Coding		IDD / SDD
Conversational, ad-hoc		Structured, specification-first
Intent lives in the developer's head		Intent lives in the document
Low overhead to start		Higher upfront investment
Hard to reproduce results	vs	Reproducible and version-controlled
No formal documentation		Formal, reviewable specs
Best for solo, fast iterations		Best for teams, production code

DIMENSION	TRADITIONAL DEV	VIBE CODING	IDD /
Planning	Requirements doc / tickets	Mental model + ad-hoc prompts	Struct instruc docun
Implementation	Human writes all code	AI generates, human reviews	AI gen from s humar valida
Reproducibility	High (deterministic)	Low (prompt-dependent)	High (is the contra
Team Scalability	High (established processes)	Low (tribal knowledge)	High (specs
Speed	Slow (manual coding)	Very fast (but fragile)	Fast (a initial invest
Quality Control	Code review + CI	Varies (often minimal)	Spec + outp valida CI
Best For	Critical systems, large teams	Prototypes, solo exploration	Produ AI-ass develo

IDD IS VIBE CODING'S ADULT FORM

IDD doesn't replace vibe coding — it **matures** it. Many teams start with vibe coding to explore ideas, then formalize their prompts into IDD specs once they decide to build for production. Think of it as: `Vibe Coding = Prototyping` → `IDD = Engineering`.

06 Real-World IDD Patterns

Pattern: Layered Specs

Break complex projects into layers of instructions that reference each other:

- **L0 — Project Spec:** Overall architecture, tech stack, conventions
- **L1 — Module Specs:** One per module (auth, billing, notifications)
- **L2 — Feature Specs:** Individual features within a module
- **L3 — Task Specs:** Atomic coding tasks (add endpoint, fix bug)
- Each level inherits constraints from the level above

Pattern: Spec-Test-Code Loop

Combine IDD with TDD for maximum confidence:

- **Step 1:** Write the instruction spec with acceptance criteria
- **Step 2:** Generate tests first from the spec (AI writes tests)
- **Step 3:** Generate implementation that passes those tests
- **Step 4:** If tests fail, refine spec or implementation
- Tests become the executable validation of your spec

Pattern: Spec as PR Template

Use the instruction document as the pull request description:

- Every PR starts with the IDD spec that generated the code
- Reviewers check code against spec, not just code quality
- Spec becomes part of the git history — full traceability
- Future developers understand *why* code was written this way

Pattern: Instruction Library

Build a reusable library of proven instruction templates:

- `auth-api.md` — Standard authentication endpoints
- `crud-service.md` — Generic CRUD service template
- `react-form.md` — Form component with validation
- `data-pipeline.md` — ETL pipeline specification
- New team members can generate production-grade code from day one

IDD IN THE DEVELOPMENT LIFECYCLE



07 Tools, Templates & Ecosystem

Tools That Support IDD Workflows

- **Cursor Rules (.cursorrules):** Project-level instruction files that persist context across sessions — the closest thing to native IDD support
- **Claude Projects (System Prompts):** Persistent project instructions that guide every interaction within a project workspace
- **GitHub Copilot Instructions:** Repository-level `.github/copilot-instructions.md` files that constrain Copilot's behavior
- **Aider Conventions:** `.aider.conf.yml` and convention files that encode project standards
- **Windsurf Memories:** Persistent project context that carries architecture decisions across sessions
- **CLAUDE.md files:** Project documentation files that Claude Code reads automatically for context

Spec Storage Best Practices

- Store specs in `/specs` or `/docs/idd` directory
- Use Markdown for portability and git-friendliness
- Name convention: `IDD-{module}-{feature}-v{version}.md`
- Version alongside code — same branch, same PR
- Add a `SPEC_INDEX.md` listing all active specs

Spec Quality Checklist

- Every requirement is **testable** (can verify pass/fail)
- No ambiguous language ("appropriate," "nice," "fast")
- Technical constraints are **specific** (versions, libraries)
- File structure is **explicit** (no room for AI improvisation)
- Edge cases and error handling are **enumerated**
- A new team member could execute the spec without additional context

AVOID OVER-SPECIFICATION

There's a balance between thorough and excessive. If your spec is 10 pages for a 50-line function, you've gone too far. The spec should be **proportional to complexity**. For simple tasks, a few bullet points suffice. Reserve detailed specs for complex, team-critical, or security-sensitive features.

08 Key Takeaways & Self-Assessment

KEY TAKEAWAYS

- **IDD/SDD** is a structured methodology where you write detailed instruction documents *before* engaging the AI — the spec is the contract between human intent and AI execution.
- The workflow follows 6 phases: `Define` → `Review` → `Execute` → `Validate` → `Refine` → `Archive`.
- A good instruction document contains: **Context, Functional Requirements, Non-Functional Requirements, Technical Constraints, File Structure, and Acceptance Criteria**.
- IDD is the **mature evolution of vibe coding**: same AI-assisted approach, but with the discipline and reproducibility needed for production software.
- Key patterns: `Layered Specs`, `Spec-Test-Code Loop`, `Spec as PR Template`, and `Instruction Library`.
- The **golden rule**: the quality of your specification directly determines the quality of the generated code. Invest in the spec, save in debugging.
- Tools like **Cursor Rules, Claude Projects, and CLAUDE.md files** provide native support for persistent, project-level instructions — the infrastructure for IDD.

SELF-ASSESSMENT QUESTIONS

Q1. What are the three main problems with ad-hoc AI prompting that IDD solves?

> ANSWER: 1) **Inconsistency** — same prompt, different results. 2) **Context loss** — AI forgets previous decisions. 3) **No collaboration** — prompts live in one developer's head, can't be shared or reviewed.

Q2. Name the 6 sections that make up a well-structured IDD instruction document.

> ANSWER: 1) Context, 2) Functional Requirements, 3) Non-Functional Requirements, 4) Technical Constraints, 5) File Structure, 6) Acceptance Criteria.

Q3. Explain the "Spec-Test-Code Loop" pattern and why it's effective.

> ANSWER: Write the instruction spec → generate tests from the spec first → generate implementation that passes those tests → iterate if tests fail. It's effective because tests serve as the executable validation of the spec, ensuring the AI's output meets the documented requirements.

Q4. How does IDD relate to Vibe Coding? Are they opposites?

> ANSWER: They're not opposites — IDD is the **mature evolution** of vibe coding. Both use AI-assisted development, but IDD adds structure, reproducibility, and team scalability. Many teams start

with vibe coding to explore, then formalize with IDD for production.

Q5. What is the "golden rule" of IDD, and what does it mean practically?

ANSWER: The golden rule: "The quality of your specification directly determines the quality of the generated code." Practically, this means ambiguous specs produce unpredictable code. Investing time in writing clear, complete, testable instructions saves significantly more time than debugging poorly-specified AI output.

Q6. Name three tools that provide native support for IDD-style persistent instructions.

ANSWER: 1) **Cursor** — .cursorrules files for project-level instructions. 2) **Claude Projects** — persistent system prompts per workspace. 3) **GitHub Copilot** — .github/copilot-instructions.md for repository-level constraints. Also: Aider conventions, Windsurf Memories, CLAUDE.md files.

Study Progress — IDD / SDD Methodology 100%

DISCLAIMER: These study notes are for educational purposes only. Methodologies and tooling evolve rapidly in the AI-assisted development space. Always verify with primary sources.

Created by Michaël Bettan · ML Certification Hub · Licensed under Apache 2.0